CS 450/535: Assignment 02

Programming Assignments (80%)

Initial Setup

- **This assignment is in C++.**
- Copy src/app/Assign01.cpp and name it **src/app/Assign02.cpp**
- Change the application name to "**Assign02**"
- Change the window title to be "Assign02: " + your SITNET ID
    - E.g., "Assign02: realemj"
- Make a copy of the vulkanshaders/Assign01 folder and name it **vulkanshaders/Assign02**
- Modify **CMakeLists.txt** by adding the following line to the end of the file:
    - CREATE_VULKAN_EXECUTABLE(Assign02)
- Make sure the program configures, compiles, and runs as-is

Requirements

Both CS 450 and 535

The goals for this assignment are to:

- **Add the normal to each vertex and use the normal as the output fragment color**
    - Add a 3D floating-point vector in ProVertex
    - Update the pipeline attribute descriptions appropriately
    - Add inputs and outputs in the vertex shader for the normal
    - Add input in the fragment shader for the normal
    - Make sure normal is normalized before use
    - Use the normal as the "color" for each fragment
        - Since the normal elements have a range from [-1,+1], some part of the color may appear black when it drops below zero.
    - NOTE: For debugging, try adding normals to your existing model from the previous assignment.
- **Comment out the creation of your existing host 3D data**
    - …although leave the declaration of **allHostMeshes**
- **Read a path to a 3D model file from the command line**
    - If argc is less than 2:
        - Use "sampleData/models/sphere.obj" as your filename path
    - Otherwise:
        - Use argv[1]

- **Load the model data with Assimp**
    - I would do this before starting up GLFW to make exiting the program easier.
    - Use the code from the "Scene Graph and Assimp" slides.
    - If the model is not loaded correctly, print the error from Assimp's importer to STDERR and exit with error code 1 (as we do in the slides).
- **Convert the model data to our host data structures → allHostMeshes**
    - Write a function: **pro::HostMesh<ProVertex> extractMeshData(aiMesh *mesh)**
        - The overall purpose of this function to grab the vertex data and triangle indices from the given aiMesh and store this information in a HostMesh struct.
        - Create a new pro::HostMesh<ProVertex> to hold the information.
        - Loop through all vertices in the aiMesh (mesh->mNumVertices):
            - Create a ProVertex.
            - Grab the vertex position information from mesh->mVertices[i] and store it in the ProVertex's position.
                - Note: mVertices is an array of aiVector3D structs, so you will have to somehow convert to glm::vec3 structs
            - Set the color of the ProVertex to any color other than (0,0,0,1) or the background.
                - Alpha values should always be 1.0f.
            - Also grab and store the vertex normals.
            - Add the ProVertex to the HostMesh's vertices list.
        - Loop through all faces in the aiMesh (mesh->mNumFaces):
            - Grab the aiFace face from mesh->mFaces[i].
            - Loop through the number of indices for this face (face.mNumIndices):
                - Add each index for the face (face.mIndices[k]) to the HostMesh's list of indices.
        - Return the HostMesh.
    - In **main()**, instead of your original HostMesh creation code:
        - Resize allHostMeshes based on the Assimp scene's number of meshes (mNumMeshes)
        - For each mesh in the Assimp scene (mMeshes with mNumMeshes):
            - Call extractMeshData to get a HostMesh from each scene->mMeshes[i] and assign it to allHostMeshes[i].
            - Topology in all cases is a triangle list.

The existing code should already convert allHostMeshes into a list of VulkanMeshes (allMeshes). It should also loop through later and render all of the VulkanMeshes.

For now, we will blindly render all models in the file; ***we will NOT deal with the scene graph in this assignment.***

CS 450 Only

Before conversion to VulkanMeshes, adjust the vertices of each model as follows:

- Scale by 0.4f
- Subtract (0.6f*i) from Y
    - This is so multiple models do not overlap
- Subtract 0.5f from Z
- Flip the Z coordinate
    - The reason for this is that we are working in a right-handed coordinate system UNTIL we get to the clip space coordinates / NDC. **We will not need this once we have a proper projection matrix later.**

CS 535 Only

Before conversion to VulkanMeshes, for each mesh:

- Calculate the mean point of all vertex positions in the mesh → **center**
- Calculate the distance furthest from the mean point (based on the vertex positions) → **extent**
- Create a scale factor that is **(0.5f / extent)**
- For each vertex in the mesh:
    - Subtract the **center**
    - Multiply by the **scale factor**
    - Rotate the vertex by -45 degrees around the Y axis and +45 degrees around the X axis. This will make the normals easier to see on the sides. This can be accomplished with the following:
        - **p = glm::vec3(glm::rotate(glm::radians(-45.0f), glm::vec3(0,1,0))*glm::vec4(p, 1.0f));**
        - **p = glm::vec3(glm::rotate(glm::radians(45.0f), glm::vec3(1,0,0))*glm::vec4(p, 1.0f));**
    - Subtract (0.6f*i) from Y
        - This is so multiple models do not overlap
    - Subtract 0.5f from Z
    - Flip the Z coordinate
        - The reason for this is that we are working in a right-handed coordinate system UNTIL we get to the clip space coordinates / NDC. **We will not need this once we have a proper projection matrix later.**
- Write a function that will join identical vertices (based on position) in the mesh:
  void joinCloseVertices(pro::HostMesh<ProVertex> &hostMesh, float error=1e-5)
    - The general objectives are:
        - Compare every vertex position with every other vertex position.
        - Combine vertices that are within ***error*** distance of each other.
            - When combining, use the average position, average color, and summed-and-normalized normal.
        - Adjust the triangle indices accordingly, but do NOT include triangles where two or more of the indices are the same

            - Otherwise, the triangle collapses to a line or a point
        - ***You should have FEWER vertices and indices than when you started.***
    - You are free to use whatever implementation you see fit, but I would offer a couple of suggestions:
        - Do NOT test this on bunny.obj as your first test (that has over 200k vertices).
        - I would approach this as producing new vertex and index lists (rather than modifying the old ones).
        - You will probably have an easier time if you keep track of the mapping of the old to the new indices (I did this as a vector<int>).
- **ONLY call joinCloseVertices() on each mesh if the user has provided a THIRD command line argument (argv[2]), which will be the error value, AND that error value is POSITIVE.**
    - HINT: Use atof() to convert the argument

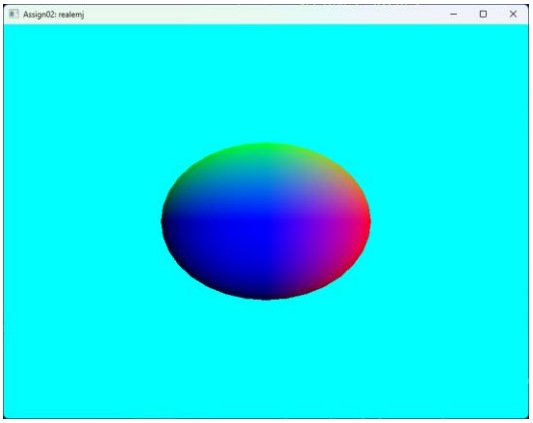
Screenshots (10%)

- **Take screenshots** of the application window and copy them to the **screenshots** folder
    - *Windows:* Alt + Screenshot button OR Windows Button + Shift + S

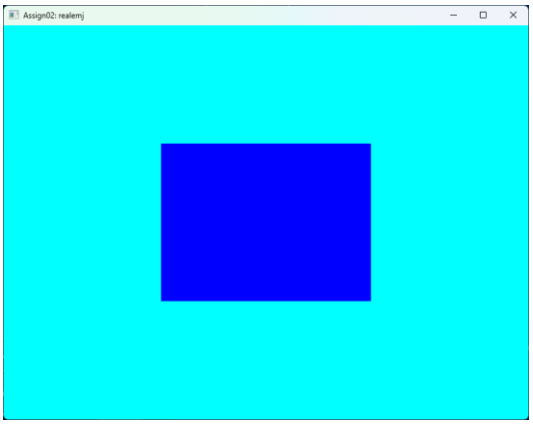
CS 450 Only

You should have **FOUR** screenshots total:

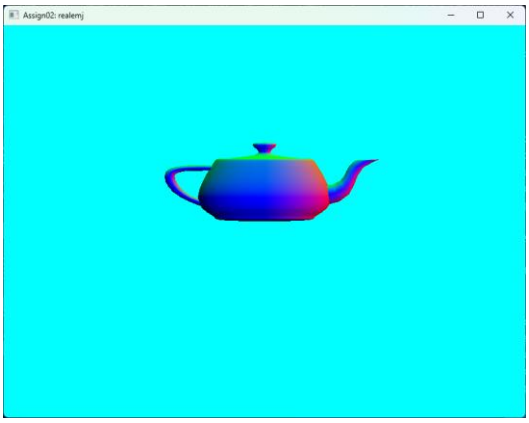
- sphere.obj → **Assign02_sphere.png**



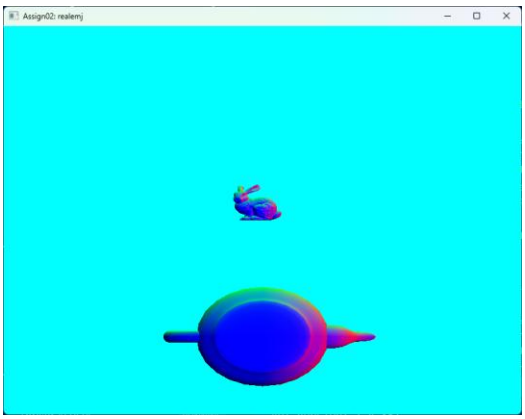
- cube.obj → **Assign02_cube.png**



- teapot.obj → **Assign02_teapot.obj**



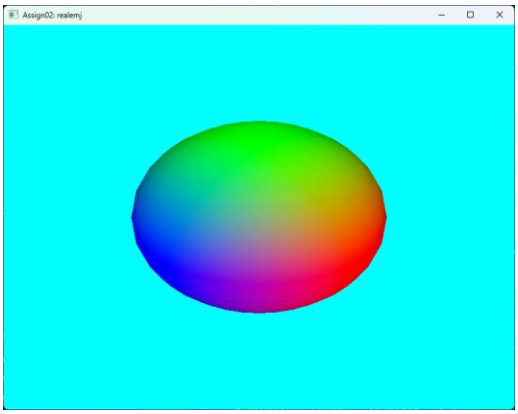
- bunnyteatime.glb → **Assign02_bunnyteatime.png**



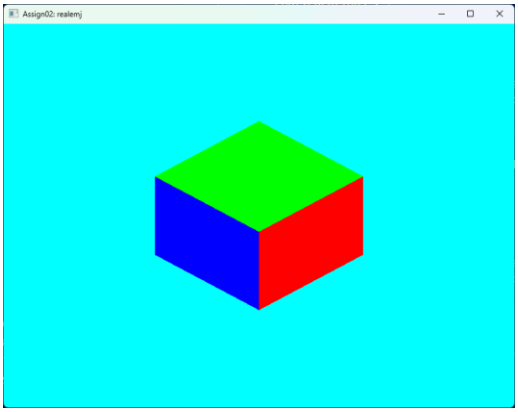
CS 535 Only

You should have **SIX** screenshots total:

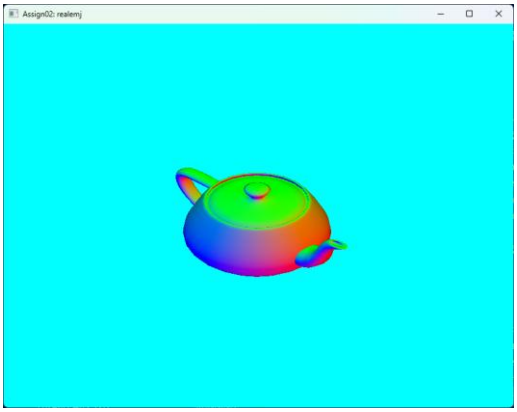
- sphere.obj (no joining) → **Assign02_sphere.png**



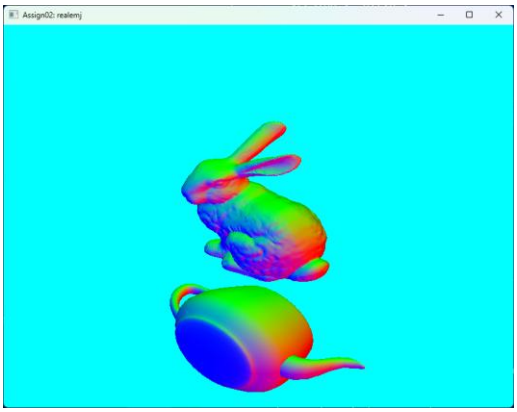
- cube.obj (no joining) → **Assign02_cube.png**



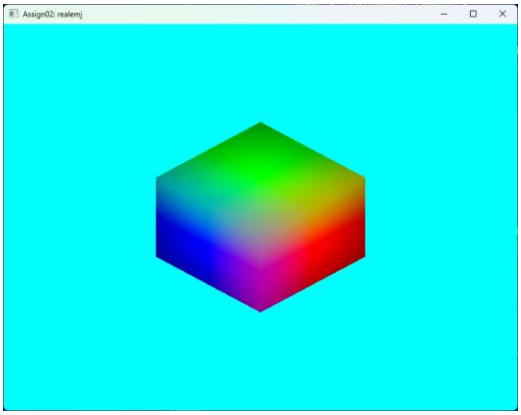
- teapot.obj (no joining) → **Assign02_teapot.png**



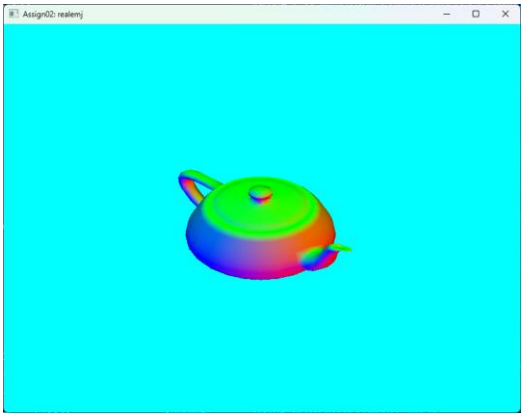
- bunnyteatime.glb (no joining) → **Assign02_bunnyteatime.png**



- cube.obj (join with 0.02 error) → **Assign02_cube_join.png**



- teapot.obj (join with 0.02 error) → **Assign02_teapot_join.png**



README (10%)

- Under the "Applications" section, add a subsection very briefly describing the assignment.
    - ***Also include your screenshots in the document as an <img> (as we have done with the exercises)***

CS 535 Only

Please VERY briefly describe your algorithm for joining the vertices.

Grading

Your OVERALL assignment grade is weighted as follows:

- 80% - Programming
- 10% - Screenshots
- 10% - README

I reserve the right to take points off for not meeting the specifications in this assignment description.

The following specific penalties (applied to the WHOLE assignment) can be expected:

Problem	**Maximum Penalty**
Code that does not compile	60
One GIGANTIC commit for whole assignment	30
Runs with Vulkan validation warnings/errors	15
Incorrect extractMeshData()	15
Mesh not rendered or positioned correctly	15
(CS 535 Only) Incorrect or poorly implemented joinCloseVertices()	15
Incorrect model loading (including default filename, flags, and error-checking)	10
Incorrect vertex shader	10
Incorrect fragment shader	10
Incorrect pipeline setup	10
README update missing/incorrect	10
Missing screenshots	10
Missing assignment Git branch	5
Failed to merge assignment branch	5
Incorrect application name	2
Incorrect window title	2
Wrong names for branch, files, etc.	2 for each
Bad commit messages	2 for each